

Trader Joe's

Final Project Write-up

COM 303

Database Systems

Spring 2026

Author: Muhammad Abdullah

Project Partner: Melanie Fernandez

May 6th 2026

1. Project Overview

This is the write-up for the final project of COM 303, Database Systems. The project's goal was to provide a realistic experience in conceptual design, logical design, implementation, operation, and maintenance of a relational database and its associated applications. My project partner Melanie Fernandez and I selected Trader Joe's as our target enterprise. We chose Trader Joe's because it has interesting properties that cover many of the harder corners of database design.

Roughly 80% of the items it sells are private label products, but it also carries a number of other branded items. It operates only physical stores plus a barebones website (which gave us a good excuse to see how an online store is structured by using Aldi as a reference). Its product catalog covers a wide variety of products from fresh produce to floral arrangements to cleaning supplies, which forced us to design a product hierarchy.

We built a fully normalized relational database in MySQL consisting of 31 tables, populated it with realistic test data covering four store locations (three physical and one online), simulated several months of checkout activity across around 30 transactions, and wrote a command-line application in Bash with multiple interfaces that lets web customers place orders, instore customers self-checkout at a kiosk, cashiers ring up purchases at a POS register, store managers monitor inventory and generate purchase orders, and corporate admins run executive analytics. We also wrote and tested a bunch of analytical queries, including all of the queries the project handout specifically asked for, to validate that the schema works the way we intended.

Many of the design choices we made look obvious in retrospect but were the result of multiple iterations and conversations between Melanie and me, and I think the reasoning behind each choice is at least as valuable as the choice itself. By the end of this project, we realised that for a database, knowing why a column exists is often more important than knowing that it exists.

2. Description of Trader Joe's

Trader Joe's is a popular chain of grocery stores in the US. It was founded in 1967 in Pasadena California and is loved by consumers for high quality, curated items, often offering international, organic, and vegetarian foods at competitive prices. Trader Joe's is owned by Aldi Nord, a German based grocery company. They are headquartered in Monrovia, California and as of early 2026, they operate over 600 locations across 42 states in the US. Over 80% of items are under the Trader Joe's brand, subjected to a rigorous tasting panel. By buying directly from manufacturers and skipping middlemen, they offer competitive pricing on certain items. Although it is sometimes described as a discount grocer, its actual business model is unusual enough that simply calling it a grocery store understates how much it differs from competitors like ShopRite, Stop & Shop, or Whole Foods.

2.1 Stores

Trader Joe's operates exclusively on physical stores. The chain does not currently run an ecommerce operation in the same way Amazon Fresh, Instacart, or even Aldi does. Its website allows customers to browse products and build a list, but does not accept orders, take payment, or schedule delivery. This was an interesting design constraint for us, because the project specification explicitly asks the database to handle a website that accepts orders, treating it as a special store with no physical location and no anonymous customers. We modeled both store types in our schema (physical and online), populated three real Trader Joe's physical store locations with data, and added one fictional Trader Joe's online store so that we could have the full set of online-store relationships and constraints. To make the online order data realistic, we used an Aldi online order receipt as our reference for what a real online grocery receipt looks like. Aldi is structurally very similar to Trader Joe's, so its receipt gave us a good idea for what columns were needed (delivery fee, service fee, gratuity, customer delivery address, etc).

Each physical store has a street address, a city, a state, a ZIP code, and operating hours. State matters in our schema because sales tax is determined by the store's state, the New York store has an 8.875% rate, the Connecticut store has 6.35%, and the Massachusetts store has 6.25%. The online store has a tax rate of 7.5% to make things simpler. We decided the 7.5% value because it is the state tax in California where Trader Joe's is headquartered. Since tax rate is a property of the location and not of the product, we made it an attribute of the `physical_store`. Operating hours are stored as a simple text attribute (e.g. "8:00AM–9:00PM Daily") because Trader Joe's stores all operate on identical hours, which makes a full opening/closing schedule an overkill.

2.2 Products

Around 80% of items on Trader Joe's are sold under the Trader Joe's Store Brand label. The remaining 20% consists of national brands, these include items like Coke and Pepsi (which we included specifically because the project handout asks for a query comparing the two), DairyPure milk, Chobani yogurt, Tropicana orange juice, Kind bars, Dole produce, Frito-Lay snacks like Doritos, General Mills cereals like Honey Nut Cheerios, Ben & Jerry's ice cream, Kellogg's frosted flakes, and others. Each product has a UPC code as its primary key, the barcode that gets scanned at checkout. We make the UPCs 15 characters in the database to standardize them across all tables (although it can be very annoying to type when testing).

A product also has a name, a size, a unit of measure (oz, lb, gallon, count, fl oz, etc), and a brand. We intentionally split size and unit of measure into separate atomic columns rather than storing "24 oz" as a single string. This was a normalization decision, storing the size as a decimal allows us to do arithmetic on it (e.g. comparing the price-per-ounce of two competing items), and storing the unit separately means we can group or filter products by unit. This may seem like a minor point, but it came up during one of our internal discussions and we agreed it mattered enough to be explicit about. Atomic attributes are also an important rule of First Normal Form.

2.2.1 Edible/Non-Edible products

Trader Joe's sells items that fall into very different categories. A pound of ground beef has dietary information, an animal species, and a cut name whereas a bottle of glass cleaner has a hazmat warning and a chemical base. Forcing both of these to share the same set of attributes would mean the Product table is full of NULL values for any given row. Every food item would have NULL hazmat warnings, and every cleaning supply would have NULL serving sizes.

To handle this cleanly, we used a disjoint specialization hierarchy. The Product entity has two mutually exclusive subtypes: Edible Product and Non-Edible Product. A given UPC belongs to one of the two. Each subtype holds the attributes that only make sense for products of that type:

- Edible Product: serving_size, calories_per_serving, dietary_certifications.
- Non-Edible Product: hazmat_warning_required, requires_assembly.

Below the Edible/Non-Edible split, we made another hierarchy level. We made eleven product subtypes that cover the items Trader Joe's sells, each with their specialized attributes:

- Edible subtypes (8): Produce, Dairy, Frozen Food, Meat & Seafood, Beverage, Snack, Premade Meal, Pantry Goods.
- Non-Edible subtypes (3): Personal Care, Floral, Cleaning Supply.

Each of these eleven subtypes has its own set of attributes. For example, the Produce subtype has produce_type (fruit/vegetable), variety (Honeycrisp, Hass Avocado, etc), and is_organic. The Dairy subtype has fat_percentage and pasteurization_type. Floral has bloom_stage and toxicity_to_pets, that last one we added because it's actually printed on Trader Joe's tulip bouquet labels and we thought it was funny enough to include.

We will discuss these in more detail with the E-R diagram.

2.2.2 Brands and Vendors

Brands are kept separate from products because many products share the same brand. The Trader Joe's Store Brand alone covers a big chunk of items in our database. A brand has a brand_id and a brand_name. The relationship between a brand and a product is one-to-many, a brand has many products, but a given product belongs to exactly one brand.

Vendors are the wholesalers that supply products to stores. The relationship between vendors and brands is many-to-many, a single vendor like Pepsico might supply Pepsi, Tropicana, Aquafina, Gatorade, Lay's, Doritos, Quaker, and other brands. Conversely, a single brand like Kind might be supplied by multiple regional vendors. Because this is many-to-many, we

modeled it with an associative entity called `Vendor_Brand_Map`. In our test data, we have six vendors (National Fresh Produce Co, Global Household Supplies, Beverage Distribution Inc, Dairy and Cold Chain Supply, Frozen Foods Wholesale, Snack Partners LLC, plus a couple added later for new product categories) and the mappings reflect realistic distribution arrangements.

2.2.3 Recursive Hierarchy

Product type has a specialization/generalization hierarchy. For example cola is a type of soda, which is a type of beverage, which is a type of food. We chose to model categories as a separate entity with a self-referential `parent_category_id` rather than as a fixed depth hierarchy of nested attributes. This means the depth of the tree is unlimited, if we ever wanted to add a “Cola” subcategory under “Soda” under “Beverage” under “Food”, nothing in the schema would need to change. In practice, our test data uses two levels, a top-level “Food” or “Non-Food” category and a single child level for the eleven concrete categories (Produce, Dairy, Frozen Food, Snack, Pantry Goods, Premade Meal, Beverage, Meat & Seafood, Bakery, Cleaning Supplies, Personal Care, Floral). Some products belong to multiple categories. Baking soda is a cleaner, a food, and a drug. So, we chose to model the product-to-category relationship as many-to-many through an associative entity called `Product_Category_Map`. Each row in this map associates one UPC with one category, so a single UPC can be linked to multiple categories. We do not currently have an item in our test data that belongs to three categories at once, but the schema supports it without modification.

2.2 Customers

There are two types of customers, those who join the frequent shopper program (which we are calling “identified” customers- with an ID) and those who walk in, pay cash, and leave without giving any personal information (“anonymous” customers). For identified customers, we store first name, last name, email, and a delivery address (street, city, state, ZIP). The delivery address is important because it’s used for online orders, the Aldi receipt we used has a “Deliver-to” field, and we wanted our online checkout to be able to populate the same field on a generated receipt.

Anonymous customers are handled simply, the `customer_id` column on the `sale_transaction` table is nullable. When a walk in pays cash, the database leaves `customer_id` as NULL. This is a deliberate choice. Some alternative designs we considered, like creating a single ‘GUEST’ customer record and pointing all anonymous transactions at it, have problems. They make it impossible to distinguish two different anonymous shoppers, which complicates any future market-basket analysis. They also make foreign-key joins behave in unexpected ways (e.g. “all transactions by customer 1” would suddenly include thousands of unrelated walk-ins). Over here, NULL is the standard relational way of saying that this attribute does not apply to this row.

The online store has no anonymous customers, every web order requires an identified customer because the system needs an address to ship to. Our schema does not currently enforce this

with a check constraint, but the application layer (the Bash interface for the web store) does require a customer ID before it allows a checkout. The application enforces a business rule that the schema permits but does not require.

3. Business rules that influenced the database design

The most important step in designing a database is to take the business rules and our own knowledge of how Trader Joe's actually operates, and convert each into a structural property of the schema. This section discusses the business rules we identified and the schema-level decisions.

3.1 Rules about Stores

- STORE is a first-class entity, not an attribute of products. Each store has a unique `store_id`.
- Some stores are physical locations, one is a website. The two kinds of stores have different attributes. Hence, STORE has two disjoint subtypes, `PHYSICAL_STORE` and `WEB_STORE`, and a given `store_id` appears in one of those two child tables.
- Sales tax depends on physical location. So, `tax_rate` is an attribute of `physical_store`, populated by state. The web store does not have a `tax_rate` column because the tax model for online orders is handled separately at checkout (in our application, we use a flat 7.5% on web orders, but in production this would be calculated from the customer's delivery address).
- Each store has its own opening hours. So, `operating_hours` is an attribute of `physical_store`. We chose a single string column rather than a separate Hours table because all Trader Joe's stores share identical hours.

3.2 Rules about Products

- Every product has a unique UPC code. So, `UPC` is the primary key of the `PRODUCT` table.
- Products come in different sizes and packaging. So `size` (decimal) and `unit_of_measure` (string) are both attributes of `PRODUCT`, split for atomicity, as discussed in 2.2.
- A variety of products may be sold under the same brand. A specific UPC belongs to exactly one brand. So, `brand_id` is a foreign key in `PRODUCT`, and `BRAND` is a separate entity. The relationship is one-to-many (one brand has many products).
- Product types form a specialization/generalization hierarchy. So, `CATEGORY` is a separate entity with a self-referencing `parent_category_id`.
- Some products belong to multiple categories. So, `PRODUCT_CATEGORY_MAP` is an associative entity (M:N) between `PRODUCT` and `CATEGORY`.

- Different products have different attributes. So, PRODUCT has a disjoint specialization into EDIBLE_PRODUCT and NON_EDIBLE_PRODUCT, which are further specialized into eleven concrete subtypes.

3.3 Rules about Inventory and Pricing

- Not all products are at all stores. So, INVENTORY is a many-to-many associative entity between STORE and PRODUCT. A row only exists for combinations that are stocked. If a store doesn't carry a product, there is no row for that (store_id, upc) pair.
- Pricing may be different at different stores. So, current_price is an attribute of INVENTORY, NOT of PRODUCT. This is a critical decision, putting it on PRODUCT would force every store to have the same price, which would be wrong.
- Each store has its own inventory level for each product. So, quantity_on_hand is an attribute of INVENTORY.
- Each store needs to decide when to reorder, and in what quantity. So, reorder_threshold (when to trigger an order) and target_stock_level (how much to order up to) are attributes of INVENTORY. These can be set differently per store. We discuss this in detail.

3.4 Rules about Customers and Transactions

- Customers may have an account, others may remain anonymous. So, customer_id on SALE_TRANSACTION is nullable for in-store transactions.
- The web store has no anonymous customers. So, enforced at the application layer, the web checkout will not proceed without a valid customer ID.
- A customer purchases a market basket of items in one transaction. So, SALE_TRANSACTION represents the whole purchase event. LINE_ITEM is an associative entity (M:N) between SALE_TRANSACTION and PRODUCT, with one row per scanned item.
- Receipts must reflect the price the customer actually paid, even if prices change later. So, unit_price_sold is an attribute of LINE_ITEM, not derived from the live INVENTORY table. We talk about this in detail later because it's one of the most important decisions in the whole project.

3.5 Rules for the Supply Chain

- Vendors sell products to stores. A vendor may sell many brands. So, VENDOR is its own entity. VENDOR_BRAND_MAP is an associative entity for the M:N relationship between vendors and brands.
- Stores generate purchase orders to restock. So, PURCHASE_ORDER is its own entity, linked to STORE and VENDOR.
- A purchase order has line items, specific products in specific quantities. So, PO_LINE_ITEM is an associative entity (M:N) between PURCHASE_ORDER and PRODUCT, which holds quantity_ordered and unit_cost.

- Vendors fill purchase orders by sending shipments with delivery dates. So, SHIPMENT is its own entity. We modeled it as a foreign key from purchase_order to shipment, allowing multiple POs to share one shipment.
- When a shipment arrives, the inventory at the destination store goes up by the quantity ordered. So, this is an application-level operation. The manager-receive-shipment workflow in our Bash app handles it inside a transaction.

3.5 Rules about Aisles and Product placement

These rules were made late in the project, after we realized that some of the queries (especially the “where is this product located in the store” lookups our cashier and kiosk interfaces want to do) needed to know the physical layout of the store.

- Each physical store has multiple aisles. The website does not have aisles. So, AISLE is its own entity, with a foreign key to PHYSICAL_STORE, not to the parent STORE table. This prevents the database from ever assigning an aisle to the website.
- A single product may be placed in more than one aisle in a store (crossmerchandising for example, fresh basil might appear both in the produce aisle and in the snacks aisle). So, PRODUCT_PLACEMENT_MAP is an associative entity (M:N) between AISLE and PRODUCT.

4. The Entity Relationship Diagram

This section walks through the conceptual design of the database, the E-R diagram, entity by entity and relationship by relationship. The diagram below shows the final state of the model after all the iterations described later.

4.2 Final ER Diagram

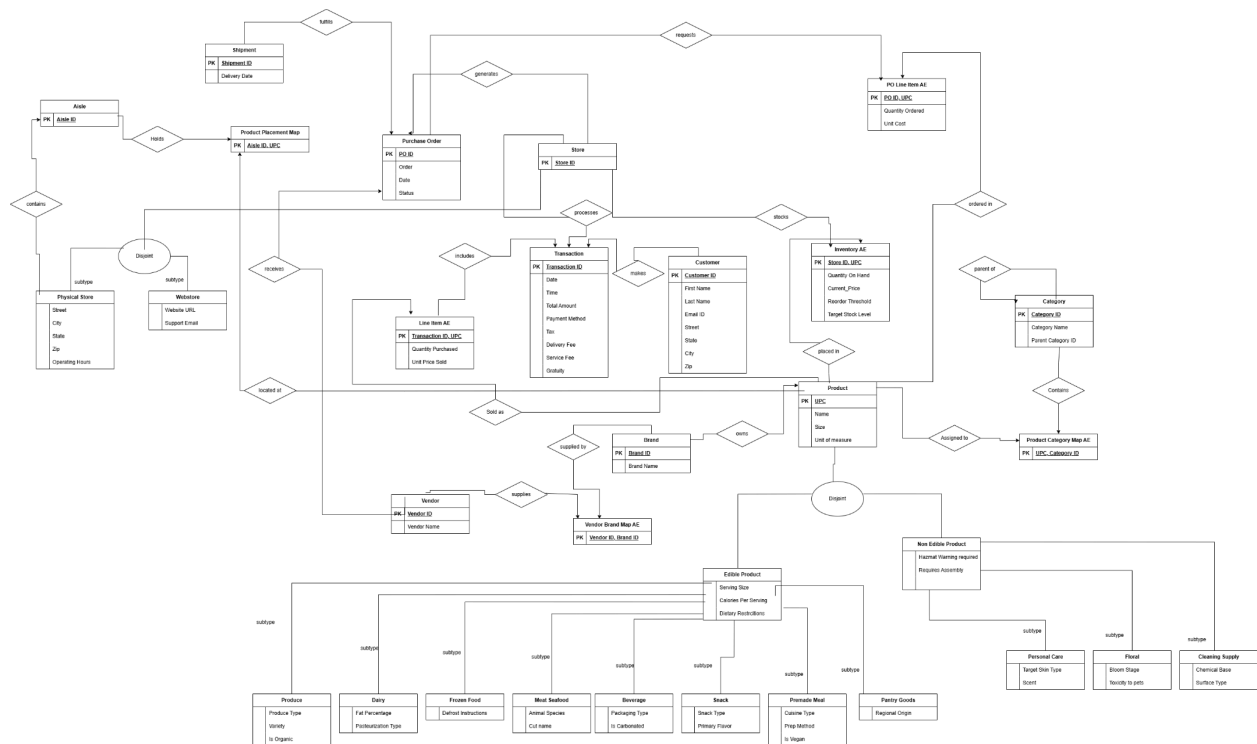


Figure 1. Final Entity-Relationship diagram for the Trader Joe's database.

4.2 Entities and their Attributes

The diagram contains 24 entities. They fall into a few categories, independent entities (which exist on their own and are uniquely identified by an internal ID), weak entities (which depend on a parent entity for their identity), and associative entities (which exist only to link two other entities in a many-to-many relationship). We have deliberately drawn the M:N associative entities as rectangles in the diagram (labeled “AE” for Associative Entity), because every M:N relationship will become a separate table in the relational schema.

4.2.1 Store Entity and its specialization

STORE has only one attribute, `store_id` (primary key). It is a parent in a disjoint specialization, meaning every store is one of two subtypes:

- PHYSICAL_STORE: street, city, state, zip, operating_hours, tax_rate
- WEB_STORE: website_url, support_email

The disjoint constraint is shown by the “Disjoint” label between the parent and the children in the diagram. A store id appears in exactly one of the two child tables. This pattern lets us cleanly

hold attributes that only apply to one kind of store, without resorting to a sea of NULL columns on a single STORE table.

4.2.2 Product Entity and its specialization

PRODUCT has the attributes UPC (PK), product_name, size, unit_of_measure, and a foreign key brand_id pointing to BRAND. Like STORE, PRODUCT is the parent of a disjoint specialization/

- EDIBLE_PRODUCT: serving_size, calories_per_serving, dietary_certifications
- NON_EDIBLE_PRODUCT: hazmat_warning_required, requires_assembly

Below this layer is a second level of disjoint specialization. EDIBLE_PRODUCT specializes into eight categories:

- PRODUCE: produce_type, variety, is_organic
- DAIRY: fat_percentage, pasteurization_type
- FROZEN_FOOD: defrost_instructions
- MEAT_SEAFOOD: animal_species, cut_name
- BEVERAGE: packaging_type, is_carbonated
- SNACK: snack_type, primary_flavor
- PREMADE_MEAL: cuisine_type, prep_method
- PANTRY_GOODS: regional_origin

And NON_EDIBLE_PRODUCT specializes into three categories:

- PERSONAL_CARE: target_skin_type, scent
- FLORAL: bloom_stage, toxicity_to_pets
- CLEANING_SUPPLY: chemical_base, surface_type

The choice of which attributes to push down into a subtype was a judgment call in some cases. For example, defrost_instructions is on FROZEN_FOOD and not on EDIBLE_PRODUCT because, even though some non-frozen items have storage instructions, defrost is a frozen-food-specific concept. By contrast, dietary_certifications is on EDIBLE_PRODUCT (the parent) because every food item could plausibly be Halal, Kosher, Vegan, Gluten Free, etc, regardless of whether it's a snack or a frozen meal.

4.2.3 BRAND, VENDOR, and CATEGORY

BRAND has brand_id (PK) and brand_name. VENDOR has vendor_id (PK) and vendor_name. Both are deliberately minimal. CATEGORY has category_id (PK), category_name, and parent_category_id (a foreign key back to category_id, allowing the category tree to nest to arbitrary depth).

4.2.4 CUSTOMER

CUSTOMER has customer_id (PK), first_name, last_name, email, street, city, state, and zip. We considered breaking the address out into its own entity, but a customer here has only one address (their delivery address), so a separate Address table would be unnecessary normalization.

4.2.5 INVENTORY

INVENTORY is an associative entity with a composite primary key of (store_id, upc) and three non-key attributes, quantity_on_hand, current_price, reorder_threshold, and target_stock_level. This is where pricing and stock data live, and the placement is intentional, pricing varies per store, so it cannot live on PRODUCT, and stock is by definition per-store.

4.2.6 SALE_TRANSACTION and LINE_ITEM

SALE_TRANSACTION has transaction_id (PK), store_id (FK), customer_id (FK, nullable), transaction_date, transaction_time, total_amount, payment_method, tax_amount, delivery_fee, service_fee, and gratuity. The split of the financial fields is necessary (discussed later). LINE_ITEM is an associative entity between SALE_TRANSACTION and PRODUCT, with composite PK (transaction_id, upc) and attributes quantity_purchased and unit_price_sold. The unit_price_sold is important, it locks in the price the customer actually paid at the moment of the sale, so that historical receipts cannot be changed when prices update.

4.2.7 SHOPPING_CART and CART_ITEM

Carts are different from transactions. A cart is an in-progress purchase that hasn't been paid for yet. SHOPPING_CART has cart_id (PK), store_id, customer_id (nullable, for guest carts at the kiosk), and a created_timestamp. CART_ITEM is the associative entity holding the items in the cart with a composite PK (cart_id, upc) and a quantity attribute. A cart isn't strictly required for a database, you could just create a transaction at checkout time and skip the cart entirely. But our application interfaces (especially the web store and the in-store kiosk) need a place to put items as the customer adds them, before checkout. The cart structure also lets us validate things like "is the requested quantity available?" before committing a transaction. Carts get cleared after checkout.

4.2.8 PURCHASE_ORDER, PO_LINE_ITEM, and SHIPMENT

PURCHASE_ORDER has po_number (PK), store_id (FK), vendor_id (FK), shipment_id (FK, nullable until the shipment is created), order_date, and status (Pending, Shipped, Delivered). PO_LINE_ITEM is the associative entity between PURCHASE_ORDER and PRODUCT, with composite PK (po_number, upc) and attributes quantity_ordered and unit_cost. SHIPMENT has shipment_id (PK) and delivery_date. It's a deliberately minimal entity. A shipment is just a delivery date that one or more POs are bound to. We considered making it more complex (carrier, tracking number, etc.) but the project doesn't require it.

4.2.9 AISLE and PRODUCT_PLACEMENT_MAP

AISLE has aisle_id (PK), store_id (FK to PHYSICAL_STORE), and aisle_identifier (a readable name like “Produce - Front” or “Dairy and Beveragews”). The store_id foreign key targets PHYSICAL_STORE rather than the parent STORE table. This choice is what makes it physically impossible for the database to assign an aisle to the website.

PRODUCT_PLACEMENT_MAP is the associative entity between AISLE and PRODUCT. Composite PK (aisle_id, upc), no other attributes. A product can appear in multiple aisles, and an aisle can contain multiple products.

4.3 Relationships and Cardinalities

Each of the relationships in the diagram has a specific cardinality.

Relationship	Cardinality	Notes
BRAND owns PRODUCT	1:N	A brand owns many products; each product has exactly one brand.
PRODUCT in PRODUCT_CATEGORY_MAP	M:N	A product can be in multiple categories; a category contains multiple products.
CATEGORY parent-of CATEGORY	1:N (recursive)	Self-referencing; each category has at most one parent, and each can have many children.
VENDOR supplies BRAND (via VENDOR_BRAND_MAP)	M:N	A vendor supplies many brands; a brand can be supplied by many vendors.
STORE stocks PRODUCT (via INVENTORY)	M:N	A store stocks many products; a product is in many stores.
PHYSICAL_STORE contains AISLE	1:N	Each aisle belongs to exactly one physical store.
AISLE holds PRODUCT (via PRODUCT_PLACEMENT_MAP)	M:N	An aisle holds many products; a product can be in many aisles.
STORE processes SALE_TRANSACTION	1:N	A store processes many transactions; each transaction occurs at exactly one store.

CUSTOMER makes SALE_TRANSACTION	1:N (optional)	A customer makes many transactions; each transaction has at most one customer (NULL for anonymous walk-ins).
SALE_TRANSACTION includes PRODUCT (via LINE_ITEM)	M:N	A transaction includes many products; a product appears in many transactions.
STORE has SHOPPING_CART	1:N	A store has many in-progress carts.
SHOPPING_CART contains PRODUCT (via CART_ITEM)	M:N	A cart contains many products; a product can be in many carts.
STORE generates PURCHASE_ORDER	1:N	A store generates many POs; each PO targets exactly one store.
VENDOR receives PURCHASE_ORDER	1:N	A vendor receives many POs from many stores.
PURCHASE_ORDER has PO_LINE_ITEM (with PRODUCT)	M:N	A PO has many line items; a product can appear in many POs.
SHIPMENT fulfills PURCHASE_ORDER	1:N	A shipment can fulfill multiple POs (batched delivery), but each PO is fulfilled by at most one shipment.
PRODUCT specializes into EDIBLE / NON_EDIBLE	Disjoint	Each UPC is one of the two.
EDIBLE_PRODUCT specializes into 8 subtypes	Disjoint	Each edible UPC is one of: Produce, Dairy, Frozen Food, Meat & Seafood, Beverage, Snack, Premade Meal, Pantry Goods.
NON_EDIBLE_PRODUCT specializes into 3 subtypes	Disjoint	Each non-edible UPC is one of: Personal Care, Floral, Cleaning Supply.
STORE specializes into PHYSICAL / WEB	Disjoint	Each store_id is one of the two.

4.4 Relationships and Cardinalities

One detail that deserves a few words on its own is the recursive parent-of relationship in CATEGORY. The instruction's example, "cola is a type of soda, which is a type of beverage, which is a type of food", is the motivation for our structure. Each row in CATEGORY has a parent_category_id column that is itself a foreign key to category_id. Top-level categories (Food, Non-Food) have NULL parent_category_id. Mid-level categories (Beverage, Snack, etc.) point to one of the top-level categories. If we wanted to add a "Soda" subcategory, we would just insert a new row with category_name = "Soda" and parent_category_id pointing at the "Beverage" row. Querying it requires a recursive expression, if we want to compute the full chain of a category, but for the simple two-level hierarchy in our test data, an ordinary JOIN of CATEGORY against itself is sufficient. We use this in Query 7 (the master department revenue rollup), which joins each child category back to its parent to roll up sales by top-level department.

5. Logical Design

This section walks through the relational schema that resulted from mapping the E-R diagram of Section 4 to MySQL tables. The mapping followed standard rules, each entity becomes a table, each many-to-many relationship becomes its own table with a composite primary key, each one-to-many relationship is implemented by placing a foreign key on the "many" side, each disjoint specialization is implemented by giving each subtype its own table whose primary key is also a foreign key to the parent table's primary key. After the basic mapping, we made a small number of refinements based on Chapter 8 normalization principles, mostly around making sure all attributes were atomic and that no transitive dependencies existed. The result is 31 tables. The full DDL is in tj.sql. Below we walk through each table, grouped by purpose, with notes on any non-obvious decisions.

5.1 Tables

These tables hold the master product catalog and its hierarchy. They are read constantly by every other part of the system.

BRAND

Column	Type	Notes
brand_id	INT AUTO_INCREMENT	Primary key. auto-increment is compact for FK use.
brand_name	VARCHAR(255) NOT NULL	E.g. "Trader Joe's Store Brand", "Coke", "Pepsi".

VENDOR

Column	Type	Notes
vendor_id	INT AUTO_INCREMENT	Primary key.
vendor_name	VARCHAR(255) NULL	NOT E.g. “National Fresh Produce Co”, “Beverage Distribution Inc.”

VENDOR_BRAND_MAP

Associative entity for the many-to-many relationship between vendors and brands.

Column	Type	Notes
vendor_id	INT NOT NULL	FK to vendor.vendor_id. Part of composite PK.
brand_id	INT NOT NULL	FK to brand.brand_id. Part of composite PK.

CATEGORY

Column	Type	Notes
category_id	INT AUTO_INCREMENT	Primary key.
category_name	VARCHAR(255) NULL	NOT E.g. “Food”, “Beverage”, “Cleaning Supplies”.
parent_category_id	INT NULL	FK to category.category_id (self-referencing). NULL for top-level categories.

The self-referencing FK is what implements the recursive hierarchy. Top-level categories like “Food” (id 10) and “Non-Food” (id 20) have parent_category_id = NULL. Subcategories like “Beverage” (id 12) have parent_category_id = 10.

PRODUCT

Column	Type	Notes
upc	VARCHAR(15) NULL	NOT Primary key. We use VARCHAR rather than INT because UPCs can have leading zeros, which would be lost in an integer column.
product_name	VARCHAR(255) NULL	NOT E.g. “Whole Milk”, “Doritos Nacho Cheese”.

size	DECIMAL(10,2)	Numeric size. Atomic with unit_of_measure.
unit_of_measure	VARCHAR(50)	E.g., "oz", "gallon", "count", "fl oz".
brand_id	INT NOT NULL	FK to brand.brand_id.

PRODUCT_CATEGORY_MAP

Associative entity for the M:N between products and categories. Each row maps one UPC to one category.

Column	Type	Notes
upc	VARCHAR(15) NOT NULL	FK to product.upc. Part of composite PK.
category_id	INT NOT NULL	FK to category.category_id. Part of composite PK.

EDIBLE_PRODUCT

Column	Type	Notes
upc	VARCHAR(15) NOT NULL	Primary key AND foreign key to product.upc.
serving_size	VARCHAR(100)	E.g. "1 cup", "8 fl oz".
calories_per_serving	INT	E.g. 150.
dietary_certifications	VARCHAR(255)	Comma-separated list, e.g. "Halal, Kosher, Vegan".

NON_EDIBLE_PRODUCT

Column	Type	Notes
upc	VARCHAR(15) NOT NULL	PK and FK to product.upc.
hazmat_warning_required	BOOLEAN FALSE	DEFAULT E.g. true for cleaning supplies with bleach.
requires_assembly	BOOLEAN FALSE	DEFAULT E.g. a small piece of furniture.

EDIBLE SUBTYPE TABLES

Each of these is a small table with a UPC primary key (also a foreign key back to edible_product.upc) plus a few subtype-specific columns.

Subtype Table	Subtype-Specific Columns
produce	produce_type, variety, is_organic
dairy	fat_percentage, pasteurization_type
frozen_food	defrost_instructions
meat_seafood	animal_species, cut_name
beverage	packaging_type, is_carbonated

snack	snack_type, primary_flavor
-------	----------------------------

premade_meal	cuisine_type, prep_method
--------------	---------------------------

pantry_goods	regional_origin
--------------	-----------------

NON-EDIBLE SUBTYPE TABLES

Subtype Table	Subtype-Specific Columns
---------------	--------------------------

personal_care	target_skin_type, scent
---------------	-------------------------

floral	bloom_stage, toxicity_to_pets
--------	-------------------------------

cleaning_supply	chemical_base, surface_type
-----------------	-----------------------------

5.2 Store Tables

STORE

Column	Type	Notes
store_id	INT AUTO_INCREMENT	Primary key. The “parent” table for the store specialization.

The base store table is minimal. It holds only the surrogate key. All real attributes are in the subtype tables.

PHYSICAL_STORE

Column	Type	Notes
store_id	INT NOT NULL	PK and FK to store.store_id.

street	VARCHAR(255) NOT NULL	Street address of the physical location.
city	VARCHAR(100) NOT NULL	City name.
state	CHAR(2) NOT NULL	Two-letter state code (e.g. “NY”).
zip	VARCHAR(20) NOT NULL	ZIP code.
operating_hours	VARCHAR(100)	Free-text hours; all TJ stores share “8AM–9PM Daily”.
tax_rate	DECIMAL(5,4) NOT NULL DEFAULT 0	Per-state sales tax rate. Added late in development.

WEB_STORE

Column	Type	Notes
store_id	INT NOT NULL	PK and FK to store.store_id.
website_url	VARCHAR(255) NOT NULL	E.g. ' https://www.traderjoes.com '.
support_email	VARCHAR(255) NOT NULL	Customer support contact.

AISLE

Column	Type	Notes
aisle_id	INT AUTO_INCREMENT	Primary key.
store_id	INT NOT NULL	FK to physical_store.store_id (NOT to store.store_id).
aisle_identifier	VARCHAR(50) NOT NULL	Human-readable names like “Dairy and Beverages”.

The fact that this FK targets physical_store.store_id and not store.store_id is the entire reason this table works correctly, it makes it impossible for any aisle to belong to the website.

PRODUCT_PLACEMENT_MAP

Column	Type	Notes
aisle_id	INT NOT NULL	FK to aisle.aisle_id. Part of composite PK.
upc	VARCHAR(15) NOT NULL	FK to product.upc. Part of composite PK.

5.3 Customer and Inventory Tables

CUSTOMER

Column	Type	Notes
customer_id	INT AUTO_INCREMENT	Primary key.
first_name	VARCHAR(100)	Nullable, frequent shoppers may decline to provide info.
last_name	VARCHAR(100)	Nullable.
email	VARCHAR(255)	Used for online order confirmations.
street	VARCHAR(255)	Delivery address for online orders.
city	VARCHAR(100)	Delivery address.
state	VARCHAR(50)	Delivery address.
zip	VARCHAR(20)	Delivery address.

INVENTORY

Column	Type	Notes
store_id	INT NOT NULL	FK to store.store_id. Part of composite PK.
upc	VARCHAR(15) NOT NULL	FK to product.upc. Part of composite PK.
quantity_on_hand	INT UNSIGNED NOT NULL DEFAULT 0	Current stock level.
current_price	DECIMAL(10,2) NOT NULL	Per-store price.
reorder_threshold	INT NOT NULL DEFAULT 10	Stock level that triggers a reorder.
target_stock_level	INT NOT NULL DEFAULT 50	Stock level we'd like to be at after restock.

The composite PK (store_id, upc) means a given product can have at most one inventory row per store. This is enforced by the database. quantity_on_hand is UNSIGNED to prevent negative inventory. The reorder logic uses both threshold and target (discussed later).

5.4 Sales / Transaction Tables

SHOPPING_CART

Column	Type	Notes
cart_id	INT AUTO_INCREMENT	Primary key.
store_id	INT NOT NULL	Which store this cart belongs to (a kiosk cart vs. a web cart).
customer_id	INT NULL	Nullable for guest carts at the kiosk.
created_timestamp	DATETIME CURRENT_TIMESTAMP	DEFAULT When the cart was started.

Our application uses three special cart_ids reserved for the three interfaces, 888 for the web store, 777 for the kiosk, and 999 for the cashier. This allows the Bash app to target a single “session cart” rather than juggling cart IDs. In a real production system, a unique cart per shopper session would be generated, but for the demo this is much simpler.

CART_ITEM

Column	Type	Notes
cart_id	INT NOT NULL	FK to shopping_cart.cart_id. Part of composite PK.
upc	VARCHAR(15) NOT NULL	FK to product.upc. Part of composite PK.

quantity INT NOT NULL

How many of these UPCs are in the cart?

SALE_TRANSACTION

Column	Type	Notes
transaction_id	INT AUTO_INCREMENT	Primary key.
store_id	INT NOT NULL	FK. Which store processed the sale.
customer_id	INT NULL	FK. Nullable for anonymous in-store purchases.
transaction_date	DATE NOT NULL	Date of sale.
transaction_time	TIME NOT NULL	Time of sale.
total_amount	DECIMAL(10,2) NOT NULL	Final total charged to the customer.
payment_method	VARCHAR(50) NOT NULL	E.g. Cash, Visa, Credit Card.
tax_amount	DECIMAL(10,2) NOT NULL DEFAULT 0	Sales tax portion of the total.

delivery_fee	DECIMAL(10,2) DEFAULT 0	NOT	NULL	\$3.99 for online orders, \$0 for in-store.
service_fee	DECIMAL(10,2) DEFAULT 0	NOT	NULL	Online platform service fee.
gratuitty	DECIMAL(10,2) DEFAULT 0	NOT	NULL	Driver tip for online orders.

The four fee columns at the bottom (tax_amount, delivery_fee, service_fee, gratuitty) all default to 0. For an in-store transaction, only tax_amount is non-zero. For a web transaction, all four are non-zero. We discuss why we ended up with this structure (rather than a single “fees” column) in more detail.

LINE_ITEM

Column	Type	Notes
transaction_id	INT NOT NULL	FK. Part of composite PK.
upc	VARCHAR(15) NOT NULL	FK. Part of composite PK.
quantity_purchased	INT NOT NULL	How many units of this UPC.
unit_price_sold	DECIMAL(10,2) NOT NULL	Price at the moment of sale. Locked in. Not a foreign key reference to inventory.current_price.

Note that unit_price_sold is its own column rather than being computed from inventory.current_price.

5.5 Supply Chain Tables

SHIPMENT

Column	Type	Notes
shipment_id	INT AUTO_INCREMENT	Primary key.
delivery_date	DATE NOT NULL	When the shipment is expected (or arrived).

PURCHASE_ORDER

Column	Type	Notes
po_number	INT AUTO_INCREMENT	Primary key.
store_id	INT NOT NULL	FK. Which store is reordering.
vendor_id	INT NOT NULL	FK. Which vendor we're ordering from.
shipment_id	INT NULL	FK. NULL until a shipment has been created to fulfill this PO.
order_date	DATE NOT NULL	When the PO was generated.
status	VARCHAR(50) NOT NULL	"Pending", "Shipped", or "Delivered".

PO_LINE_ITEM

Column	Type	Notes
po_number	INT NOT NULL	FK. Part of composite PK.
upc	VARCHAR(15) NOT NULL	FK. Part of composite PK.
quantity_ordered	INT NOT NULL	How many units ordered.
unit_cost	DECIMAL(10,2) NOT NULL	What we're paying per unit (wholesale).

5.6 Indexes and constraints

The schema declares the following indexes beyond the primary keys (which are automatically indexed in MySQL).

The reasoning for each.

- `idx_sale_transaction_store` improves "all transactions at this store" lookups (used by the manager dashboard and several analytics queries).
- `idx_sale_transaction_customer` accelerates "show me my order history" lookups (used by the customer interface).
- `idx_sale_transaction_date` accelerates date-range queries like "top 5 stores so far this year" and "top selling products in March 2026."
- `idx_line_item_upc` accelerates "how many units of UPC X have we sold" aggregations (used by every top-selling-product query).
- `idx_inventory_upc` accelerates the inverse: "which stores carry UPC X?" This is used by the corporate price adjuster, which has to update prices across all stores at once.
- `idx_physical_store_state` accelerates the "top selling products in each state" query.
- `idx_shopping_cart_store` and `idx_shopping_cart_customer` accelerate cart lookups during checkout and "my open carts" lookups.

Foreign key constraints are declared inline in every CREATE TABLE that references another table. We have not used ON DELETE CASCADE anywhere, for a real retail database we'd want deletions to be very rare and explicit (we'd never want to delete a customer record and have all their historical transactions disappear), so we used the default restrict behavior.

6. Problems we encountered and how we solved them

This section is pretty long and detailed. It includes the real iterations we went through. The early decisions that turned out to be wrong, the bugs that revealed modeling problems, and the redesigns we ended up making. The bulk of the work in this project happened in this section. The problems are presented in roughly the order we encountered them.

6.1 Treating Physical and Online Stores as the Same Entity

In our initial draft of the schema, STORE was a single table that tried to hold every attribute that any store might have: street, city, state, zip, operating_hours, website_url, support_email, and an is_online_store boolean. The intent was to use the boolean to figure out which fields were real for a given row. if is_online_store = TRUE, the website_url and support_email columns are populated and the address columns are NULL. If is_online_store = FALSE, the address columns are populated and the URL columns are NULL. Looking at our initial ER diagram from before the redesign, we can see this single-table approach.

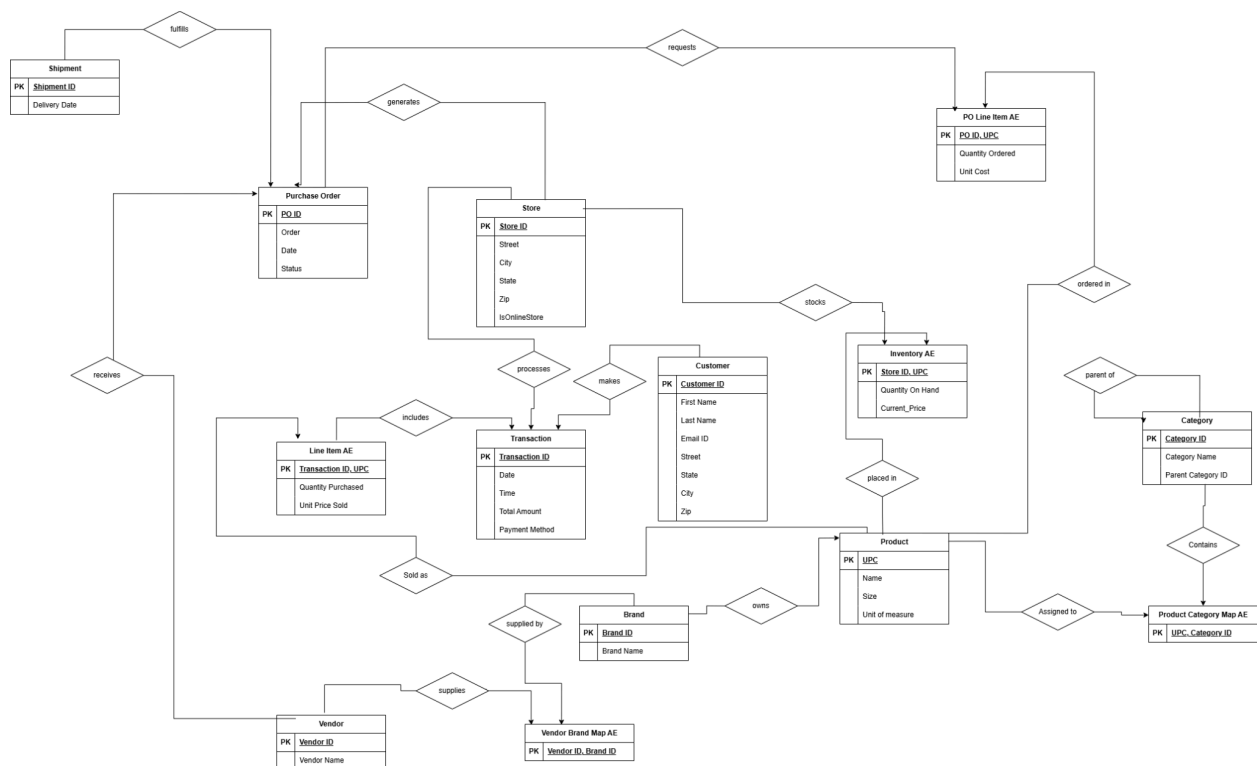


Figure 2. Initial Entity-Relationship diagram for the Trader Joe's database.

This worked, in the sense that we could populate it and run queries on it. But it had several problems that became increasingly annoying as we wrote more queries.

- Half of every store row was NULL. Every web store had NULL for street, city, state, zip, and operating_hours. Every physical store had NULL for website_url and support_email. NULL is meaningful in SQL, using it to mean "this attribute doesn't apply" is the wrong way to use it. NULL really means "unknown," not "not applicable."
- Constraints were hard to express. We wanted to enforce that, for example, every physical store has a non-NULL address, but we couldn't add a NOT NULL constraint on the street because that would break web stores. So we'd have to handle this in application code, which is fragile.

We replaced the single STORE table with three, a base STORE table holding only the store_id, a PHYSICAL_STORE child table holding the physical attributes, and a WEB_STORE child table holding the online attributes. Each child table's primary key is also a foreign key to STORE.store_id. A given store_id appears in exactly one child table, the disjointness.

- No more NULL columns. Every column on every row holds a real value.
- Constraints can be added per subtype. PHYSICAL_STORE.street is NOT NULL; WEB_STORE.website_url is NOT NULL; neither breaks the other.
- Queries naturally express what they care about. "List all cities" joins against PHYSICAL_STORE only. "Show me the websites" joins against WEB_STORE only.
- Adding another store kind is a new table, not a new column. Schema migrations stay localized.

We now need a JOIN whenever we want both store-level info, the store_id and any base-table columns, and the subtype-specific info. In practice this has not been a problem, since we know up front whether we're looking at physical or online operations.

6.2 Aisles Were Connected to All Stores, Including Online

Once we introduced the AISLE entity (we needed it for the cashier and kiosk interfaces, which want to display aisle locations to help customers find products), we initially connected it to STORE, meaning the foreign key on AISLE.store_id pointed at the parent STORE table. But, a website doesn't have aisles. We changed AISLE.store_id from being a foreign key against store.store_id to a foreign key against physical_store.store_id.

6.3 Products in multiple categories

Our first instinct was to put a category_id column directly on the PRODUCT table, making the relationship between product and category one-to-many. A banana belongs to Produce, a Coke belongs to Beverage and so on. But we wanted a single UPC to belong to multiple categories e.g. baking soda as mentioned in the instructions. So, we introduced PRODUCT_CATEGORY_MAP, an associative entity with composite primary key (upc, category_id) that allows a single UPC to map to as many categories as needed.

To say baking soda is a cleaner, food, and drug, we'd insert three rows, (baking_soda_upc, "Cleaning Supplies"), (baking_soda_upc, "Food"), (baking_soda_upc, "Drug").

None of our actual test data uses a product that's in multiple categories, but the schema is ready for it.

6.4 Receipts whose numbers don't add up

Our initial `sale_transaction` table had a single `total_amount` column and that was all. The plan was that the total would equal the sum of `(line_item.quantity_purchased * line_item.unit_price_sold)` plus tax. Then we tried it with the Aldi online order receipt.

It broke the total down into:

- Item Total: \$52.03
- Delivery: \$3.99
- Service Fee: \$3.64
- Discount: \$0.00
- Gratuity: \$5.64
- Subtotal: \$65.30
- Due Total: \$65.30

If we ran `SUM(quantity_purchased * unit_price_sold)` over the line items of this transaction, we'd get \$52.03. But the `total_amount` in our schema was \$65.30. The receipt has \$13.27 of fees that aren't represented anywhere in `line_items`. We expanded `sale_transaction` to have four dedicated fee columns, `tax_amount`, `delivery_fee`, `service_fee`, and `gratuity`. Each of these defaults to 0. For an in-store transaction, only `tax_amount` is set.

After we made this fix, we ran an SQL update to recalculate `total_amount` across all existing test transactions, just to make sure the invariant held everywhere.

```
UPDATE sale_transaction st JOIN (select transaction_id, sum(quantity_purchased *
unit_price_sold) as actual_product_total from line_item group by transaction_id) li_totals on
st.transaction_id = li_totals.transaction_id set st.total_amount = li_totals.actual_product_total +
st.tax_amount + st.delivery_fee + st.service_fee + st.gratuity;
```

6.5 Historical Price Locking

This was perhaps the most important conceptual issue we hit. In our first version of the schema, `line_item` didn't have a `unit_price_sold` column. Instead, we computed the price of a line item by joining against `inventory.current_price` at query time. We thought that since `current_price` was already in `inventory`, storing it again on `line_item` would be a redundant denormalization.

This reasoning is wrong. `inventory.current_price` is a present column, it tells you what the price is right now. But a receipt is past, it tells you what the customer paid at the moment of sale. If a price changes between the sale and the present, those two numbers should be different. With a `current_price`-based design, every historical receipt in the database changes whenever a price changes. Receipt issued at \$4.99 last March, price went up to \$5.49 last week, now your March receipt also says \$5.49 in the database. This is bad:

- It's wrong on the face of it. The customer paid \$4.99. The receipt should say \$4.99.
- It breaks tax math. Tax was calculated against the \$4.99 price, if the receipt now shows \$5.49, the tax doesn't match the line item.

We added `unit_price_sold` as a column on `line_item`. When a transaction is created, the application reads `inventory.current_price` at that exact moment and writes the value into `line_item.unit_price_sold`. After that, `line_item.unit_price_sold` is permanent. Nothing in the schema or application updates it, even if the price in inventory changes later.

With this column in place, several things become true and stay true regardless of what happens to inventory.

- Receipts always show the price the customer paid.
- Total revenue queries can be answered correctly without knowing what current prices are.
- Comparing "price paid" to "current price" becomes a useful analytical query. One of our admin dashboard queries does this.

6.6 Reorder logic wasn't sufficient

Our initial inventory table only had `quantity_on_hand` and `current_price`. So when we thought about how to implement it, we ran into a question, how does the script know an item is low? Hard-coding a threshold in the application would be wrong, different products have different sell-through rates, and the reorder threshold for milk should be very different from the threshold for cleaning supplies. Setting one global threshold for the whole store would mean over-ordering on slow movers and stocking out on fast movers. Even if we had a threshold, the script also has to decide how much to order. Just enough to get back above the threshold? Twice the threshold? Some fixed multiple? Without a target value somewhere in the database, the script would have to guess.

So, we added two columns to inventory, `reorder_threshold` (the level below which a reorder should be triggered) and `target_stock_level` (the level we'd like to be at after the reorder is delivered). Both can be set independently per (store, product) pair, which lets a store manager tune them as they learn the local sell-through rate.

We use this logic in two places: the manager's "low inventory alerts" view and the manager's "generate purchase order" workflow. The threshold and target are also surfaced in our test data with realistic values. Milk has a low threshold (15) but a moderate target (60-80) because it sells fast and spoils. Bottled water has a higher threshold and a much higher target because it sells in bulk. Smaller stores have lower targets than the (fictional) online warehouse, which has a target stock level in the hundreds for everything.

6.7 Missing State Taxes

Tax math is interesting. The receipt I used for testing the in-store experience was from a New York City Trader Joe's, and the tax line on it was "\$1.29 @ 8.875%". That number is not a

constant, it depends on which state and sometimes city the store is in. Our New London Connecticut store should use 6.35% sales tax. Our Boston store should use 6.25%. Our online store has 7.5% (Tax Rate in California).

Hard-coding the tax rate would be wrong. Putting it on every transaction would be redundant. Putting it on every product-store inventory row would be redundant. The right place is the store itself. We added a `tax_rate` column to `physical_store` with type `DECIMAL(5,4)`. All three of our physical stores were updated with the correct rate for their state.

The cashier checkout flow in our application reads the tax rate from `physical_store` at checkout time and uses it to compute the tax on the cart. The web store uses a flat 7.5% rate, a real implementation would compute it from the customer's delivery address, but it was too complicated for this project.

6.7 Cross Merchandising

In a real Trader Joe's, certain products appear in more than one aisle. Avocados are mostly in the produce section but are sometimes also placed near the chips and salsa to suggest a guacamole pairing. Cookies might be placed near both the snack aisle and the bakery aisle. Cross-merchandising is an intentional retail strategy.

Our initial design had `product_id` as a column on `aisle` (or, equivalently, `aisle_id` as a column on `product`). Either way, the relationship was one-to-one between aisle and product. This meant a single product could only live in one aisle, which is wrong for the cross-merchandising case.

We made the relationship many-to-many through `PRODUCT_PLACEMENT_MAP`, an associative entity with composite key (`aisle_id`, `upc`). A product can now appear in any number of aisles. In our test data, avocados appear both in the produce aisle and in the snacks-and-pantry aisle for this cross-merchandising reason. The cross-merchandising audit query, lists all products that appear in more than one aisle in a given store.

6.9 Was the Premade Meal “is_vegan” Redundant?

During schema cleanup, we noticed that the `PREMADE_MEAL` subtype had a column called `is_vegan`. But the parent `EDIBLE_PRODUCT` table already has a `dietary_certifications` column that includes "vegan" as one of its possible values. Having both means we could express the same fact two ways. We dropped `premade_meal.is_vegan`. From now on, vegan-ness is recorded only in `dietary_certifications`. We also dropped a `discount_amount` column from `sale_transaction` that we'd added but never populated (Trader Joe's famously does not run discounts in the way that other grocers do, every customer gets the same price every day).

6.10 Determining Which Vendor a Restock Should Go To

When a manager triggers a reorder for a specific product UPC, the system has to decide which vendor to send the PO to. This is non-trivial because the relationship between products and

vendors is indirect, a product has a brand, and a brand is supplied by one or more vendors (the vendor_brand_map). So “who do I order this from?” (Realized this while presenting in class).

Our Bash app's manager-generate-PO workflow uses the following resolution chain when the manager picks a UPC to restock:

```
SELECT v.vendor_id FROM product p JOIN vendor_brand_map vbm ON p.brand_id =  
vbm.brand_id JOIN vendor v ON vbm.vendor_id = v.vendor_id WHERE p.upc = "$order_upc"  
LIMIT 1;
```

This finds the first vendor that supplies the product's brand, and uses that vendor for the PO. The LIMIT 1 is a deliberate simplification. For products whose brand has multiple authorized vendors, we just pick one. A real system would do something smarter, compare wholesale prices, check delivery time, prefer the vendor with the next scheduled delivery, and so on. For the project this is sufficient and demonstrates that the relationship chain works.

7. Implementation

The project specification asks for several types of users to interact with the database, each with different needs, a database administrator who runs SQL directly, market analysts running OLAP queries, online customers placing orders, an automatic reorder generator, vendors handling shipments, and cashier registers ringing up purchases. We built a single Bash command-line application with five role-based sub-menus, each tailored to one of these user types. This kept the implementation effort manageable while still demonstrating that the database supports each kind of user. The application is a single shell script (app.sh) that uses the MySQL command-line client to talk to the database. We chose Bash because the spec explicitly says a simple command-line interface will suffice if your Web and/or GUI skills are not up to the challenge, and because Melanie had experience working in Bash. I also picked it up pretty quickly. A real consumer-facing web store would obviously be built differently, but the database underneath would be the same.

Top Level Menu

Picking 1 through 5 routes to the appropriate sub-menu. Each sub-menu has its own session state, for example, the cashier menu prompts for a store ID at login because a register lives in exactly one store, the manager menu does the same, the admin menu requires a PIN. The web customer menu requires a customer ID because online orders cannot be anonymous.

Webstore (for customers)

Browsing lets them filter by department (a category from the category table), shows available products with their current online price, and lets them add items to a cart by entering a UPC and quantity. The cart is stored under cart_id = 888, which is reserved for the web session. The system enforces that the requested quantity does not exceed the online inventory level. If a customer tries to add more than is in stock, they get an error.

Checkout asks the customer to pick a tip percentage (0%, 10%, 15%, or 20%), then displays the receipt: subtotal, tax (7.5% flat for online), delivery fee (\$3.99), driver tip, and grand total. Typing "PAY" commits the order. Internally, this:

- Re-validates that no item in the cart exceeds inventory, in case another customer just bought the last unit between cart-build and checkout.
- Inserts a new `sale_transaction` with all the fee columns populated and `store_id = 4` (the web store).
- Inserts one `line_item` per `cart_item`, locking in the `current_price` into `unit_price_sold`.
- Decrements `inventory.quantity_on_hand` by the purchased quantities.
- Clears the cart.

All five of those steps run inside a single MySQL transaction (BEGIN/COMMIT) so that if any of them fail, none of them take effect, preventing the database from ending up in a partially committed state where, for example, a transaction record exists but inventory was not decremented. This is a crucial property. Real-world e-commerce systems get this wrong all the time and it leads to phantom orders or oversold inventory.

The "view my order history" option runs a simple JOIN of `sale_transaction` X `line_item` X `product`, filtered by the `customer_id`, and shows what the customer bought when. This is the kind of personal-history feature that any decent online store has.

Instore self checkout

This interface simulates a self-checkout kiosk in a physical store. Login asks for the kiosk's `store_id` (1, 2, or 3), and optionally a customer ID for frequent shoppers. Leaving the customer ID blank logs in as a guest, which is the anonymous-customer case.

The kiosk uses `cart_id = 777` and shows aisle locations along with each product. This is what makes the kiosk useful in a real store. When a customer scans an item they did not intend to buy and wants to put it back, the kiosk can tell them where to put it. The aisle lookup is implemented as a JOIN against `product_placement_map` and `aisle`, scoped to the kiosk's `store_id`, with `GROUP_CONCAT` to handle the cross-merchandising case. A product in two aisles shows both.

Checkout uses the per-state tax rate from `physical_store.tax_rate`. The CT kiosk applies 6.35%, the NY kiosk applies 8.875%, and the MA kiosk applies 6.25%.

Cashier POS

Same general shape as the kiosk, but framed as a cashier-operated register. The cashier logs in with a store ID, then scans items into `cart_id = 999`, and at checkout asks the customer if they have a frequent-shopper ID (optional, and we accept invalid IDs by silently treating them as guests). Tax is calculated the same way as the kiosk, via the `physical_store.tax_rate` column. The cashier flow is closer to the workflow shown on the actual Trader Joe's receipt I used as a reference (the New York store, with 8.875% NY state sales tax). The receipt format we display

at the end of checkout mimics the structure of a real Trader Joe's receipt with a customer name (or Guest), a list of line items with their per-line subtotals, a state tax line, and a grand total.

Store Manager

This is the operational backend for a store manager. Login asks for a store ID (the manager logs in to their specific store).

Low Inventory Alerts

Lists every (store, product) pair in this store where `quantity_on_hand <= reorder_threshold`. Sorted ascending by stock level, the items in the most urgent need of restocking show up first.

Generate Purchase Order

Shows suggested reorder quantities (`target_stock_level - quantity_on_hand`) for every item below target. The manager picks a UPC and enters a quantity. The system then:

- Looks up an authorized vendor for the product via the `vendor_brand_map` chain (Section 6.11).
- Creates a SHIPMENT record with `delivery_date` set to three days from today.
- Creates a PURCHASE_ORDER referencing the chosen vendor and the new shipment, with `status = "Pending"`.
- Creates a PO_LINE_ITEM with the requested quantity and a placeholder `unit_cost` of 1.50. A real system would resolve this from a vendor pricelist, we simplified.

All four steps run inside a single transaction so that no partial PO ever exists in the database.

Receive Pending Shipments

Shows pending POs awaiting delivery, along with their expected delivery date. The manager picks a PO that has just arrived and the system:

- Updates the PO status from "Pending" to "Delivered".
- Increments `inventory.quantity_on_hand` by the `quantity_ordered` for each PO line item.

After this runs, the store's inventory reflects the newly-arrived stock.

Cross-Merchandising Audit

Lists every product in this store that appears in more than one aisle. We added this because we wanted a way to verify the `PRODUCT_PLACEMENT_MAP` many-to-many relationship was working correctly, and because cross-merchandising is a real retail concern that managers actually look at for restocking, expiry date etc.

Corporate Admin Dashboard

This is the most query-heavy of the interfaces. Login requires a hard-coded PIN (1234, for the demo). The admin dashboard has eleven options:

- Top Selling Products Overall (company-wide unit sales ranking).
- Highest Performing Stores (Physical & Online), by fiscal year, with a UNION across physical stores and the web store.
- Brand Head-to-Head, pick any two brand names (e.g. Coke vs. Pepsi) and see who's winning by city. This is the query the project handout asks for.
- Web Store Revenue Breakdown, separates pure product revenue from collected fees (tax, delivery, service, gratuity).
- Global Price Adjuster, let the admin update a product's price across every store at once. Used to demonstrate that historical receipts are immune to price changes.
- Historical Price Variance Audit, list every line_item where unit_price_sold differs from the current inventory.current_price. This is the proof that historical price locking works.
- Master Department Revenue Rollup, uses the recursive category hierarchy to roll up sales by top-level department (Food vs. Non-Food).
- Top 20 Products by Specific Store, the project instruction's required query.
- Top 20 Products by State, also required.
- Market Basket Analysis, given a product (e.g. Milk), what are the top three categories of items also bought in the same transaction?
- Logout.

There's also a per-checkout overselling check. Just before commit, the code re-checks that no item in the cart exceeds the current inventory level. If another customer's checkout has already run and depleted an item between the time we built the cart and the time we tried to commit, our checkout fails with an error message rather than silently overselling. This is a simplified version of the optimistic concurrency control that real e-commerce systems use.

8. Sample Queries and their justifications

This section walks through our test queries one by one. Each is presented with the SQL we wrote, an explanation of what it does, and the business reason a Trader Joe's manager or analyst might want to run it. The first five queries are the ones the project handout explicitly listed as "queries managers may find of interest"; the rest are queries we added either to demonstrate specific schema features (like historical price locking) or because they're useful in their own right.

All queries below run successfully on the populated database. Output is omitted for space, but representative results were verified during testing.

Q1: Top 20 Top-Selling Products at Each Store

Required by the project specification. This query is asked by store managers who want to know what is selling well at their store, so they can make sure those items stay in stock.

```
select p.upc, p.product_name, sum(li.quantity_purchased) as total_units_sold
```

```

from sale_transaction st

join line_item li on st.transaction_id = li.transaction_id

join product p on li.upc = p.upc

where st.store_id = ?

group by p.upc, p.product_name

order by total_units_sold desc

limit 20;

```

In our admin dashboard this is shown as "Top 20 Products by Store," which prompts the user for a store ID and runs the query against that store. The composite index on line_item(transaction_id, upc) and the per-store index on sale_transaction(store_id) keep this fast even with millions of transactions.

Q2: Top 20 Top-Selling Products in Each State

Also required. This is the regional equivalent of Q1 and would be of interest to a regional buyer trying to figure out what to push in (say) the New England market vs. the West Coast market.

```

select ps.state, p.upc, p.product_name, sum(li.quantity_purchased) as total_units_sold

from sale_transaction st

join physical_store ps on st.store_id = ps.store_id

join line_item li on st.transaction_id = li.transaction_id

join product p on li.upc = p.upc

where ps.state = ?

group by ps.state, p.upc, p.product_name

order by total_units_sold desc

limit 20;

```

This joins through physical_store rather than through store, which means online orders are excluded, sensible because online orders are not tied to a single state. If a regional analyst wanted online orders included, they'd run a separate query that joins through web_store.

Q3: Top 5 Stores with the Most Sales So Far This Year

Required. The version below is parameterized by year so it can be run for any fiscal year:

```
select 'physical' as store_type, ps.city as location, ps.state, sum(st.total_amount) as total_sales
from sale_transaction st
join physical_store ps on st.store_id = ps.store_id
where year(st.transaction_date) = ?
group by st.store_id, ps.city, ps.state

union
```

```
select 'online' as store_type, 'web store' as location, 'n/a' as state, sum(st.total_amount) as
total_sales
from sale_transaction st
join web_store ws on st.store_id = ws.store_id
where year(st.transaction_date) = ?
group by st.store_id, ws.website_url
order by total_sales desc
limit 5;
```

The union is necessary because physical and online stores live in different child tables. There is no single table that gives us location info for both kinds of store, so we have to query each separately and merge the results. The "n/a" filler in the state column for the online row is a small concession to keeping the columns aligned across the two halves of the union.

Q4: In How Many Stores Does Coke Outsell Pepsi?

Required (the example in the handout). Our admin dashboard generalizes this to any two brands the user types in:

```
select ps.city,
sum(case when b.brand_name = 'Coke' then li.quantity_purchased else 0 end) as coke_sold,
```

```

sum(case when b.brand_name = 'Pepsi' then li.quantity_purchased else 0 end) as pepsi_sold,
case
    when sum(case when b.brand_name = 'Coke' then li.quantity_purchased else 0 end)
        > sum(case when b.brand_name = 'Pepsi' then li.quantity_purchased else 0 end)
    then 'Coke wins'
    when sum(case when b.brand_name = 'Coke' then li.quantity_purchased else 0 end)
        < sum(case when b.brand_name = 'Pepsi' then li.quantity_purchased else 0 end)
    then 'Pepsi wins'
    else 'tie'
end as market_leader
from sale_transaction st
join line_item li on st.transaction_id = li.transaction_id
join product p on li.upc = p.upc
join brand b on p.brand_id = b.brand_id
join physical_store ps on st.store_id = ps.store_id
where b.brand_name in ('Coke', 'Pepsi')
group by ps.city;

```

This uses the SUM(CASE ...) pivot trick to put the two brands' totals in side-by-side columns for each city, then a third case expression to compute the winner. The result is a per-city report that immediately makes it clear where each brand has dominance.

Q5: Top 3 Categories Customers Buy in Addition to Milk

Required (the example in the handout was "top 3 types of product that customers buy in addition to milk"). This is a market-basket-analysis query.

```

select p2.product_name as bought_with_milk, c.category_name, count(*) as times_together
from line_item li1

```

```

join sale_transaction st on li1.transaction_id = st.transaction_id

join product p1 on li1.upc = p1.upc

join line_item li2 on st.transaction_id = li2.transaction_id

join product p2 on li2.upc = p2.upc

join product_category_map pcm on p2.upc = pcm.upc

join category c on pcm.category_id = c.category_id

where p1.product_name like '%Milk%'

and p1.upc != p2.upc

group by p2.product_name, c.category_name

order by count(*) desc

limit 3;

```

The double join of line_item to itself is the standard pattern for this kind of co-occurrence query. We use the transaction_id as the bridge to find every other product that appeared in the same basket as milk.

Q6: Master Department Revenue Rollup (Hierarchical)

This query exercises the recursive category hierarchy.

```

select parent.category_name as master_department,

concat('$', format(sum(li.quantity_purchased * li.unit_price_sold), 2)) as total_revenue

from line_item li

join product p on li.upc = p.upc

join product_category_map pcm on p.upc = pcm.upc

join category child on pcm.category_id = child.category_id

join category parent on child.parent_category_id = parent.category_id

group by parent.category_name

```

```
order by sum(li.quantity_purchased * li.unit_price_sold) desc;
```

This shows that the recursive parent_category_id structure allows us to do this rollup with a single ordinary join.

Q7: Historical Price Variance Audit

This query proves the historical price locking works.

```
select st.transaction_date, p.product_name,  
li.unit_price_sold as locked_receipt_price,  
i.current_price as live_inventory_price,  
(i.current_price - li.unit_price_sold) as price_inflation_difference  
from sale_transaction st  
join line_item li on st.transaction_id = li.transaction_id  
join product p on li.upc = p.upc  
join inventory i on st.store_id = i.store_id and li.upc = i.upc  
where li.unit_price_sold != i.current_price  
order by st.transaction_date desc  
limit 15;
```

Whenever an admin runs the "Global Price Adjuster" option, this audit query will start showing results.

Q8: Web Store Revenue Breakdown (Pure Product vs Fees)

```
select ws.website_url,  
sum(tx_totals.product_rev) as pure_product_revenue,  
sum(st.tax_amount + st.delivery_fee + st.service_fee + st.gratuity) as total_fees_collected,  
sum(st.total_amount) as total_gross_revenue
```

```

from web_store ws
join sale_transaction st on ws.store_id = st.store_id
join (
    select transaction_id,
           sum(quantity_purchased * unit_price_sold) as product_rev
    from line_item
    group by transaction_id
) tx_totals on st.transaction_id = tx_totals.transaction_id
group by ws.store_id, ws.website_url;

```

Q9: Cross-Merchandising Audit

```

select p.product_name,
count(ppm.aisle_id) as number_of_aisles_placed,
group_concat(a.aisle_identifier separator ' & ') as exact_aisle_locations
from physical_store ps
join aisle a on ps.store_id = a.store_id
join product_placement_map ppm on a.aisle_id = ppm.aisle_id
join product p on ppm.upc = p.upc
where ps.store_id = ?
group by p.upc, p.product_name
having count(ppm.aisle_id) > 1;

```

Q10: Low Inventory Alerts

```

select i.store_id, p.product_name, i.quantity_on_hand, i.reorder_threshold
from inventory i
join product p on i.upc = p.upc

```

where i.store_id = ?

and i.quantity_on_hand <= i.reorder_threshold

order by i.quantity_on_hand asc;

Q11: Suggested Reorder Quantities

select p.upc, p.product_name, b.brand_name,

i.quantity_on_hand, i.target_stock_level,

(i.target_stock_level - i.quantity_on_hand) as suggested_order_qty

from inventory i

join product p on i.upc = p.upc

join brand b on p.brand_id = b.brand_id

where i.store_id = ?

and i.quantity_on_hand < i.target_stock_level

order by suggested_order_qty desc;

Q12: Pending Shipments Tracker

select po.po_number, po.order_date,

coalesce(s.delivery_date, 'tbd') as expected_arrival,

v.vendor_name, p.product_name, poli.quantity_ordered

from purchase_order po

join po_line_item poli on po.po_number = poli.po_number

join product p on poli.upc = p.upc

join vendor v on po.vendor_id = v.vendor_id

left join shipment s on po.shipment_id = s.shipment_id

where po.store_id = ?

and po.status = 'pending';

Q13: Top Selling Products Company-Wide

```
select p.upc, p.product_name,  
       sum(li.quantity_purchased) as total_units_sold,  
       sum(li.quantity_purchased * li.unit_price_sold) as total_revenue  
from product p  
  
join line_item li on p.upc = li.upc  
  
group by p.upc, p.product_name  
  
order by total_units_sold desc  
  
limit 15;
```

Q14: Customer Order History

```
select st.transaction_date, p.product_name,  
       li.quantity_purchased, li.unit_price_sold  
from sale_transaction st  
  
join line_item li on st.transaction_id = li.transaction_id  
  
join product p on li.upc = p.upc  
  
where st.customer_id = ?  
  
order by st.transaction_date desc;
```

Q15: Online Catalog Browse

```
select p.upc, p.product_name,  
       concat(p.size, ' ', p.unit_of_measure) as unit_size,  
       i.current_price as price,  
       coalesce(ep.dietary_certifications, 'n/a') as diet_certs,  
       case  
         when pr.is_organic = 1 then 'yes'
```

```

    when pr.is_organic = 0 then 'no'
    else 'n/a'
end as organic
from inventory i
join product p on i.upc = p.upc
left join edible_product ep on p.upc = ep.upc
left join produce pr on p.upc = pr.upc
where i.store_id = 4
and i.quantity_on_hand > 0;

```

Q16: In-Store Catalog with Aisle Locator

```

select p.upc, p.product_name,
concat(p.size, ' ', p.unit_of_measure) as unit_size,
i.current_price as price,
coalesce(ep.dietary_certifications, 'n/a') as diet_certs,
group_concat(distinct a.aisle_identifier separator ' & ') as located_in
from inventory i
join product p on i.upc = p.upc
left join edible_product ep on p.upc = ep.upc
join product_placement_map ppm on p.upc = ppm.upc
join aisle a on ppm.aisle_id = a.aisle_id
where i.store_id = ?
and a.store_id = ?
and i.quantity_on_hand > 0
group by p.upc, p.product_name, p.size, p.unit_of_measure,

```

```
ep.dietary_certifications, i.current_price;
```

Q17: Identify Products at More Than One Location

```
select p.upc, p.product_name,  
  
count(distinct i.store_id) as num_stores  
  
from product p  
  
join inventory i on p.upc = i.upc  
  
group by p.upc, p.product_name  
  
having count(distinct i.store_id) > 1  
  
order by num_stores desc;
```

9. Test data generation

Portions of the test data, primarily the bulk INSERT statements for products, inventory, transactions, and line items, were drafted with the assistance of a large language model. We provided the schema, business rules, and realism constraints we wanted, and reviewed every generated row by hand to verify foreign keys, receipt totals, and confirm the edge cases were intact. The LLM accelerated data entry, the design of the data and its validation were ours.

Item	Count	Notes
Stores	4	3 physical (NY, CT, MA) + 1 online webstore
Aisles	15	5 aisles per physical store; none for the webstore
Brands	15+	Trader Joe's Store Brand, Coke, Pepsi, DairyPure, Clorox, Windex, Chobani,

Tropicana, Kind, Dole, Frito-Lay, General Mills, Ben & Jerry's, Kellogg's, plus a few additions

Vendors	8	National Fresh Produce Co., Global Household Supplies, Beverage Distribution Inc., Dairy and Cold Chain Supply, Frozen Foods Wholesale, Snack Partners LLC, National Meat Providers, Sweet Snacks Co.
Categories	13+	Two top-level (Food, Non-Food) and concrete categories: Produce, Beverage, Dairy, Frozen Food, Snack, Premade Meal, Pantry Goods, Cleaning Supplies, Personal Care, Floral, Meat & Seafood, Bakery
Products	36	Real Trader Joe's products plus the brand-name products carried in stores
Customers	5	Identified frequent shoppers from various states
Inventory rows	~100	Most products are stocked in multiple stores at meaningfully different prices
Sale transactions	~30+	Spread across roughly two years (2024–2026) for time-series queries
Line items	~75+	Items per transaction varies from 2 to 10, mimicking real basket sizes
Purchase orders	5	Mix of Pending, Shipped, and Delivered statuses

10. Edge Cases we built in

We deliberately included several edge cases in the test data so that our queries would be exercised against the corner cases they are supposed to handle, not just the happy path.

- Roughly half of our in-store transactions have `customer_id = null`, showing cash walk-ins. This ensures any aggregate query that joins the customer needs to handle the null case correctly, or, if it does not, returns wrong results that we can detect.
- We have several inventory rows with `quantity_on_hand = 0` to make sure "available products" queries (like Q15 and Q16) properly filter those out, and to ensure the kiosk and web checkout reject orders for out-of-stock items.
- At least 5–6 inventory rows have stock at or below their threshold, which makes the manager's low-inventory alert query return non-empty results immediately on the first run.
- A product placed in two aisles at every physical store: avocados appear in both the produce aisle and the snacks-and-pantry aisle, so the cross-merchandising query has something to find. Marinara sauce also has a double-placement at one store.
- Every web transaction in our data has a `delivery_fee`, `service_fee`, and `gratuity`, so the "web store revenue breakdown" query has something interesting to split on.
- Transactions where the locked unit price differs from the current inventory price: we populated several line items at slightly older prices, then made the "current" price in inventory slightly higher. The historical price audit query immediately reveals the difference, demonstrating that the price-locking mechanism works.
- Brands that span multiple vendors: the Trader Joe's Store Brand is supplied by several vendors (National Fresh Produce, Frozen Foods Wholesale, Snack Partners), forcing the auto-resolve-vendor logic in the manager-generate-PO workflow to actually pick one rather than always returning the same answer.
- "Meat & Seafood" was added late in the project and has only a few products. This tests that aggregate queries do not crash on small-population categories.
- We do not have a baking-soda-style item in three categories, we do exercise the `m:n product_category_map` relationship with realistic single-category mappings, and the schema is ready for multi-category mappings the moment we want them.

11. Summary

This project was an end-to-end design and implementation of a relational database for a retailer modeled on Trader Joe's. We started from the project specifications and the actual operational behavior of the chain (gathered both from real receipts and from public knowledge of how Trader Joe's operates), translated business rules into entities and relationships, and implemented the resulting schema in MySQL. We then populated it with realistic test data, wrote

and tested a range of analytical queries, and built a multi-role command-line application that demonstrates how each kind of user (web customer, in-store kiosk shopper, cashier, store manager, corporate admin) interacts with the database.

The final database has 31 tables, supports four stores (three physical and one online) with their per-store inventory and pricing, contains 36 products across 12 conceptual categories, and has been exercised against approximately 30 transactions and over a dozen queries. The application logic, checkout, reorder generation, shipment receipt, low-inventory alerts, market basket analysis, and several others, runs against this schema cleanly without resorting to NULL-juggling or redundant data.

This is how the data flows. We start from the store supertype, which establishes the existence of a retail entity. This branches into two distinct subtypes, `physical_store` and `web_store`. Inside each physical store, the floor plan is divided into specific aisle records, where goods will eventually be placed. Before an item can be placed on a shelf, it must be defined in the Catalog. A universal product record is created, containing data like UPC, name, size, and unit of measure. Because grocery items are vastly different, this product filters down a specialization/generalization hierarchy. It is classified as either an `edible_product` or a `non_edible_product`. From there, it moves to a highly specific tables, such as `dairy`, `frozen_food`, or `cleaning_supply` etc. Simultaneously, the product is organized for the user interface. It is linked via the `product_category_map` to a category. Because the category table is recursive (`parent_category_id`), the product belongs to a nested hierarchy (e.g. Honeycrisp Apple -> Produce -> Food).

The product is then pushed to the local stores. The `product_placement_map` assigns the UPC to specific aisles (allowing for items like avocados to be cross-merchandised in multiple aisles). The product is then logged into the inventory table. The inventory table acts as a junction between the Store and the Product, establishing the local `current_price`, the `quantity_on_hand`, and the `reorder_threshold`. When a customer walks into a store (or logs into the Web Store), they initiate an active session by generating a `shopping_cart`. As they browse, items are temporarily added to the `cart_item` table. At this stage, the data is volatile, if the customer walks away, the cart can be flushed without impacting financial records. However, once the customer pays, the transaction becomes permanent. The database generates a `sale_transaction` (locking in the date, time, total amount, and taxes). The items are pulled from the cart and placed into the `line_item` table. The `line_item` captures the exact `unit_price_sold` at that specific second, ensuring that if corporate changes the price of milk tomorrow, today's historical receipts remain perfectly accurate. Finally, the items are mathematically deducted from the inventory table.

As customers continue to buy items, the local `quantity_on_hand` eventually drops below the `reorder_threshold`. This triggers the Store Manager's dashboard. To order more, the database must figure out who manufactures the item. The product checks its `brand_id`, which is passed through the `vendor_brand_map` to find all third-party vendors authorized to supply that brand. The Store Manager selects a vendor and generates a `purchase_order` (listing the items in `po_line_item`). To physically move the goods, the purchase order generates a new shipment

record, generating a future `delivery_date`. The purchase order remains in a "Pending" status until the truck arrives, at which point the manager receives the shipment, the status can be changed to "Delivered," and the new goods are added back into the inventory table, starting the entire cycle over again.

12. Reflection

The project had two parts that felt very different to me. The first part, coming up with the conceptual design and the relational schema, was familiar territory. Going through the project specification and translating each business rule into a structural property of the schema was almost mechanical. The second part, actually working with the schema as we wrote application code and queries, was where the real learning happened, and it was harder than I expected. Every problem mentioned is a place where the on paper design didn't work well in real implementation.

Working with a partner was useful in a way I didn't expect. Melanie and I split the work in roughly the natural way. We did almost all the design work together, and several of the trickiest decisions came out of conversations where one of us would propose something and the other would point out a case where it didn't work (mostly during the TA sessions we hosted, if no student showed up).

13. Acknowledgement

This project was completed in collaboration with my partner Melanie Fernandez. The design decisions were all a result of joint conversations.